Alfredo Arnaiz
Mar. 9, 2026
Foundations of Python Programming
Assignment 7
https://github.com/ARArnaiz/IntroToProg-Python-Mod07

Assignment 7: Classes and Objects

Introduction

This document details the process of preparing the required seventh assignment for the Foundations of Python Programming (IT FDN 110 GP Wi 26) course. This assignment builds on the learnings of the first six assignments adding the creation of custom classes to manage data and reinforcing concepts of object-oriented programming.

As in the previous assignments, this one also requires the preparation of a short program following specific criteria regarding acceptance, testing and submission. To accomplish this task, it was required to

1. read the module notes and watch the listed videos
2. read and watch additional materials related to this assignment
3. posting the relevant files to Canvas and GitHub
4. posting the link to the GitHub site into the Discussion board, and
5. perform a peer review.

Script/Program Preparation

Given the instructions and acceptance criteria, this is the process I followed:

- Review and renaming of the starter file *Assignment07-Starter.py*.
- Updated the header information.
- Set up and review module imports, constants and variables.
- Set up two new custom classes:
    - *Person*: This class includes its contructor, setter and getter methods dealing with its properties corresponding to first name and last name, plus an override of the *__str__()* method to return its data in a comma-separated format.

    - *Student*, a subclass of *Person*: This class includes its constructor, which inherits the first and last name data from its parent class and adds a new course name property, setter and getter methods dealing with its new property, plus an override of the *__str__()* method to return all its data in a comma-separated format.
- Adjusted previously existing methods to use the *Student* class instead of the previously used dictionary data structure. This applied to the following function/methods:
    - *read_data_from_file()* – The list of dictionary records read via the *json.load()* method were mapped into objects of the *Student* class and stored in the usual list data structure.
    - *write_data_to_file()* – The list of *Student* objects is serialized as a list of dictionaries and saved into the file using *json.dump()*.
    - *output_student_courses()* – This method Iterates through the list of *Student* objects and prints a formatted string of each student's first name, last name, and course name using the overriding *__str__()* method from the *Student* class. Here I experimented with multiple options until I realized that the *print()* method is the one being overridden.
    - *input_student_data()* – This method validates that name fields contain only alphabetic characters via the *Student* class property setters. It also handles unexpected errors.
- Prepared and adjusted the descriptive document strings for the classes and functions.

In this assignment, the setup of TODO comments made much clearer the changes required to adapt the script to the use of the new classes.

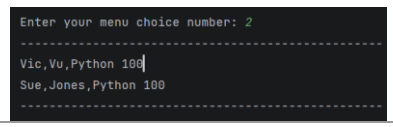
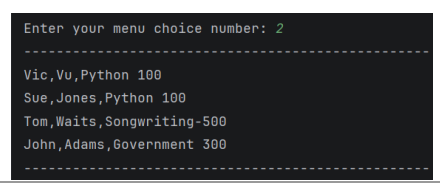
Testing

Besides testing as I modified the script, I tested full runs on

- PyCharm and
- PyCharm's terminal (equivalent to command prompt testing).

In both cases, I did full passes over the menu options multiple times considering the common sequences users may do. Here, I tested all outputs including the JSON file (see table 1 for JSON file contents).

Table 1 - Input and output (file and console)

File contents	JSON file + console output (captures from PyCharm)
Input	 
Output after adding two records	 

No unexpected results.

As last time, I followed this testing with a validation of the main cases for the error handling, see table 2 below:

Table 2 - Error messages

Test	Results (capture from PyCharm Terminal)

No input file	`Error: There was a problem with reading the file.`
	
	`-- Technical Error Message --`
	`[Errno 2] No such file or directory: 'Enrollments.json'`
	`File not found.`
	`<class 'FileNotFoundError'>`

Empty input file	`Error: There was a problem with reading the file.`
	
	`-- Technical Error Message --`
	`Expecting value: line 1 column 1 (char 0)`
	`Subclass of ValueError with the following additional properties:`
	
	`msg: The unformatted error message`
	`doc: The JSON document being parsed`
	`pos: The start index of doc where parsing failed`
	`lineno: The line corresponding to pos`
	`colno: The column corresponding to pos`
	
	`<class 'json.decoder.JSONDecodeError'>`

Wrongly formatted input file	`Error: There was a problem with reading the file.`
	
	`-- Technical Error Message --`
	`Expecting ',' delimiter: line 7 column 3 (char 86)`
	`Subclass of ValueError with the following additional properties:`
	
	`msg: The unformatted error message`
	`doc: The JSON document being parsed`
	`pos: The start index of doc where parsing failed`
	`lineno: The line corresponding to pos`
	`colno: The column corresponding to pos`
	
	`<class 'json.decoder.JSONDecodeError'>`

Invalid input for first name	`Enter the student's first name: T0m`
	`This value is not the correct type of data!`
	
	`-- Technical Error Message --`
	`The first name should not contain numbers.`
	`Inappropriate argument value (of correct type)`
	`<class 'ValueError'>`

Invalid input for last name	`Enter the student's first name: Tom`
	`Enter the student's last name: Wait2`
	`This value is not the correct type of data!`
	
	`-- Technical Error Message --`
	`The last name should not contain numbers.`
	`Inappropriate argument value (of correct type).`
	`<class 'ValueError'>`

Invalid output file (read-only)	`Error: There was a problem with writing to the file.`
	`Please check that the file is not open by another program.`
	
	`-- Technical Error Message --`
	`[Errno 13] Permission denied: 'Enrollments.json'`
	`Not enough permissions.`
	`<class 'PermissionError'>`

Summary

This document summarizes the preparation, implementation, and testing of Assignment 7 for the Foundations of Python Programming course. The assignment introduces object-oriented programming through the creation of *Person* and *Student* custom classes and updates existing functions to use class-based data instead of dictionaries, including JSON

file handling. Comprehensive testing confirmed correct functionality, validation, and error handling with no unexpected results.